中国科学院大学
《计算机组成原理(研讨课)》实验报告

姓名 王浚沣 学号 2024K8009909018 专业 计算机科学与技术
实验项目编号 4 实验名称 高速缓存(Cache)设计

注 1:	撰写此 Word 格式实验报告后以 PDF 格式保存 SERVE CloudIDE 的 `/home/serve-ide/cod-lab/reports` 目录下 (注意: reports 全部小写)。文件命名规则: `prjN.pdf`, 其中 `prj` 和后缀名 `pdf` 为小写, `N` 为 1 至 4 的阿拉伯数字。例如: `prj1.pdf`。PDF 文件大小应控制在 5MB 以内。此外, 实验项目 5 包含多个选做内容, 每个选做实验应提交各自的实验报告文件, 文件命名规则: `prj5-projectname.pdf`, 其中 "-" 为英文标点符号的短横线。文件命名举例: `prj5-dma.pdf`。具体要求详见实验项目 5 讲义。
注 2:	使用 `git add` 及 `git commit` 命令将实验报告 `PDF` 文件添加到本地仓库 master 分支, 并通过 `git push` 推送到实验课 SERVE GitLab 远程仓库 master 分支(具体命令详见实验报告)。
注 3:	实验报告模板下列条目仅供参考, 可包含但不限定如下内容。实验报告中无需重复描述讲义中的实验流程。

完整实验流程包括设计、开发、调试三个步骤:

• **设计**步骤为在代码编写前的准备阶段, 主要包括 Cache 结构选择、状态机划分、替换策略和写策略的确定, 并绘制整体数据通路。

• **开发**步骤为在 S-IDE 集成开发环境中进行 Verilog 代码编写, 实现 I-Cache、D-Cache 以及 tag/data array 等模块。

• **调试**步骤包括本地仿真验证后推送至云平台进行 FPGA 原型验证, 以及后续根据波形和访存结果进行迭代修改的全过程。

一、 设计成果(对设计流程的总结。需附上设计结构图, 可用 PPT 或者在线绘图软件绘制逻辑结构框图后保存为 PDF, 再插入到 LaTeX 中)

1 整体 Cache 结构设计

本次实验旨在为 CPU 设计独立的指令 Cache 和数据 Cache, 从而降低 CPU 访问外部存储器时的平均等待时间。与前几次实验中 CPU 直接通过握手信号访问存储器不同, 本次设计在 CPU 和内存之间加入了一个新的层次。CPU 发出的取指或访存请求先进入 Cache, 如果命中, 则直接从 Cache 内部的数据阵列中返回结果; 如果未命中, 则由 Cache 向外部内存发起 burst 读请求, 将一整行数据取回后再返回给 CPU。

按照要求, 我在本次实验中采用了四路组相联结构。Cache 一共包含 8 个 set, 每个 set 有 4 个 way, 每一行长度为 256 bit, 也就是 32 Byte。由于 CPU 每次取指或访存的基本单位为 32 bit, 因此一行 Cache 中可以保存 8 个 word。地址划分上, 我将地址低 2 位视为字节内偏移, 由于接口保证 4 字节对齐, 所以主要使用地址的 [4:2] 作为行内 word 偏移, [7:5] 作为 set index, [31:8] 作为 tag。这样的划分比较直接, 也与 tag_array 和 data_array 的地址宽度相匹配。

在替换策略上, 我没有实现复杂的 LRU, 而是为每一个 set 保存一个两位的 replace_way 指针。发生缺失时, 如果该 set 中仍存在无效 way, 则优先选择第一个无效 way; 如果四路都已经有效, 则按照 replace_way 指示的 way 进行替换, 并在 refill 完成后将该指针加一。这种近似轮转的方式实现较为简单, 虽然不能严格反映最近使用情况, 但对于本实验的规模已经能够完成基本的缓存替换功能。

2 I-Cache 设计

I-Cache 的功能相对单纯，只需要处理 CPU 的取指请求，因此我将其设计为只读 Cache。I-Cache 接收到 CPU 发来的指令地址后，先锁存请求地址，并进入 tag 读取阶段。随后比较四路 tag 和 valid 位，如果命中，则根据地址中的 word 偏移从 256 bit cache line 中选出对应的 32 bit 指令，并返回给 CPU。如果没有命中，则进入缺失处理流程。

缺失处理流程包括选择替换路、向内存发出读请求、接收 8 个 word 的返回数据、写入 tag/data array 和返回目标指令几个部分。由于 I-Cache 不会被 CPU 写入，因此不需要 dirty 位，也不需要写回阶段。这个特点使 I-Cache 的状态机比 D-Cache 更短，主要包括 S_WAIT、S_TAG_RD、S_CACHE_RD、S_RESP、S_EVICT、S_MEM_RD、S_RECV 和 S_REFILL 等状态。

我认为 I-Cache 设计中比较关键的地方是 refill 数据的组装。内存每次返回 32 bit 数据，而 Cache line 宽度为 256 bit，因此需要用 recv_cnt 记录当前收到的是第几个 word，并将其写入 refill_line 的对应位置。等到 from_mem_rd_rsp_last 有效时，说明一整行数据已经接收完成，此时才能进入 S_REFILL 状态，同时写 tag 和 data array。

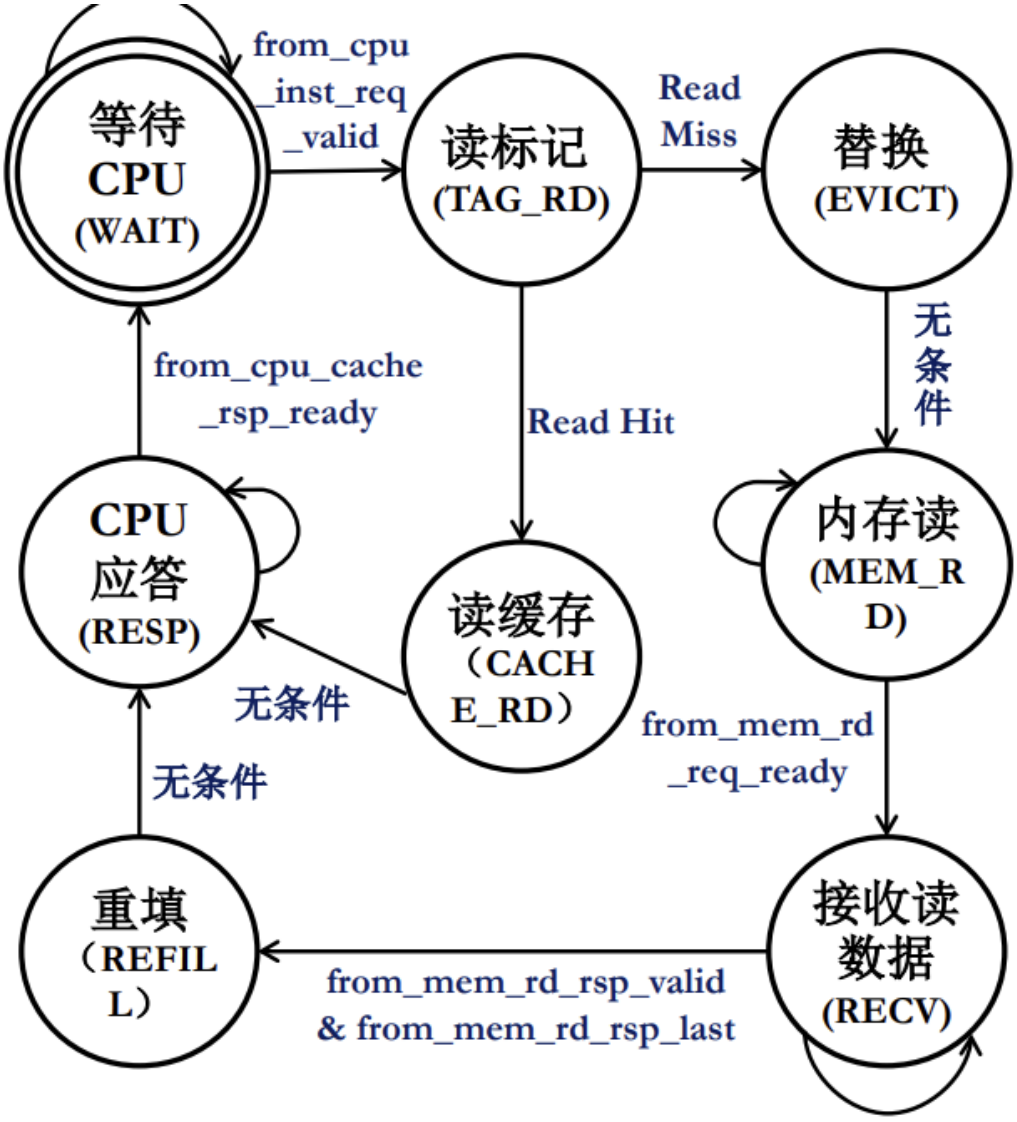


图 1: 状态转移图

```
wire hit_way0 = valid_way0[req_index] && (tag_way0 == req_tag);
wire hit_way1 = valid_way1[req_index] && (tag_way1 == req_tag);
wire hit_way2 = valid_way2[req_index] && (tag_way2 == req_tag);
wire hit_way3 = valid_way3[req_index] && (tag_way3 == req_tag);

wire cache_hit = hit_way0 || hit_way1 || hit_way2 || hit_way3;

wire [`LINE_LEN - 1:0] hit_line =   hit_way0 ? data_way0 :
                                    hit_way1 ? data_way1 :
                                    hit_way2 ? data_way2 :
                                               data_way3;
```

图 2: I-Cache 命中判断代码

3 D-Cache 设计

D-Cache 的设计比 I-Cache 更复杂，因为它需要同时支持读请求和写请求，并且还需要处理 I/O 访问。本次我采用的是阻塞式、写回、写分配的数据 Cache。阻塞式意味着 D-Cache 在处理当前请求时不会接收新的 CPU 请求；写回意味着命中写只修改 Cache 中的数据并置 dirty 位，不立即写回外部内存；写分配意味着写缺失时也会先把对应 cache line 读入 Cache，再合并写入数据。

对于普通可缓存地址，D-Cache 会先读 tag 并判断是否命中。如果读命中，则直接从命中的 cache line 中选出对应 word 返回；如果写命中，则通过写掩码将 CPU 写入的数据与原有 word 合并，再写回对应 way 的 data array，同时置 dirty 位。为了支持 sb、sh 等带字节写掩码的访存指令，我设计了 merge_word 函数，根据 wstrb 分别控制四个字节是否更新，再由 write_line 函数将合并后的 word 放回 256 bit cache line 的对应位置。

```
function [31:0] merge_word;
    input [31:0] old_word;
    input [31:0] new_word;
    input [3:0]  wstrb;
    begin
        merge_word[ 7: 0] = wstrb[0] ? new_word[ 7: 0] : old_word[ 7: 0];
        merge_word[15: 8] = wstrb[1] ? new_word[15: 8] : old_word[15: 8];
        merge_word[23:16] = wstrb[2] ? new_word[23:16] : old_word[23:16];
        merge_word[31:24] = wstrb[3] ? new_word[31:24] : old_word[31:24];
    end
endfunction

function [`LINE_LEN - 1:0] write_line;
    input [`LINE_LEN - 1:0] line;
    input [2:0] word_index;
    input [31:0] wdata;
    input [3:0]  wstrb;
    reg [`LINE_LEN - 1:0] next_line;
    begin
        next_line = line;
        case (word_index)
            3'd0: next_line[ 31:  0] = merge_word(line[ 31:  0], wdata, wstrb);
            3'd1: next_line[ 63: 32] = merge_word(line[ 63: 32], wdata, wstrb);
            3'd2: next_line[ 95: 64] = merge_word(line[ 95: 64], wdata, wstrb);
            3'd3: next_line[127: 96] = merge_word(line[127: 96], wdata, wstrb);
            3'd4: next_line[159:128] = merge_word(line[159:128], wdata, wstrb);
            3'd5: next_line[191:160] = merge_word(line[191:160], wdata, wstrb);
            3'd6: next_line[223:192] = merge_word(line[223:192], wdata, wstrb);
            3'd7: next_line[255:224] = merge_word(line[255:224], wdata, wstrb);
        endcase
        write_line = next_line;
    end
endfunction
```

图 3: 相应函数代码

当 D-Cache 发生缺失时，首先选择 victim way。如果 victim way 有效并且 dirty 位为 1，则需要先将旧 cache line 写回内存。写回阶段按 8 个 word 依次发送数据，写掩码固定为 4'b1111，直到最后一个 word 发送完成后才进入读缺失行的阶段。如果 victim way 无效或不脏，则可以直接向内存发起读请求。读回新行后，如果原请求是写请求，还需要先把 CPU 写入的数据合并到 refill line 中，再写入 Cache 并置 dirty 位。

此外，D-Cache 中还需要单独处理 I/O 或不可缓存地址。我在代码中通过地址高位判断是否需要 bypass。当访问属于 bypass 范围时，请求不会进入 tag 比较和 Cache refill 流程，而是直接走单拍读写通道。这样做可以避免外设寄存器被错误缓存，也能保证 volatile 类访问看到的是外设当前状态。

```
case (state)
    S_WAIT: begin
        if (from_cpu_mem_req_valid) begin
            req_write <= from_cpu_mem_req;
            req_addr  <= from_cpu_mem_req_addr;
            req_wdata <= from_cpu_mem_req_wdata;
            req_wstrb <= from_cpu_mem_req_wstrb;

            if ((from_cpu_mem_req_addr[31:5] == 27'b0) || (from_cpu_mem_req_addr[31:30] != 2'b00)) begin
                state <= from_cpu_mem_req ? S_BYPASS_WR_REQ : S_BYPASS_RD_REQ;
            end else begin
                state <= S_TAG_RD;
            end
        end
    end
```

图 4: 旁路判断代码

二、 开发成果(对代码开发流程的总结。需附上包含注释的 Verilog HDL 关键代码段、相应信号的仿真波形和信号变化的说明)

1 tag/data array 开发

底层存储阵列分为 tag_array 和 data_array 两类。tag_array 的数据宽度为 24 bit,地址宽度为 3 bit,正好对应每一路 8 个 set 中保存的 tag;data_array 的数据宽度为 256 bit,同样使用 3 bit 地址,用于保存每个 set 对应的 cache line。两者都采用同步写、异步读的结构,写入在时钟上升沿发生,读取则直接由 raddr 索引输出。

我在顶层 Cache 中分别实例化四份 tag_array 和四份 data_array,对应四路组相联结构。这样命中判断时可以并行读取四路 tag 和 data,减少了额外的选择等待。写入时则通过 victim_way 或 hit_way 控制具体哪一路的写使能有效。

2 I-Cache 开发

I-Cache 的主要开发工作集中在状态机和命中/缺失处理逻辑上。请求阶段, 当 from_cpu_inst_req_valid 有效且 Cache 处于 S_WAIT 状态时,我将请求地址保存到 req_addr 中。之后状态机进入 S_TAG_RD,根据 req_index 读出四路 tag,并结合 valid 位得到 hit_way0 到 hit_way3 四个命中信号。如果有任意一路命中,则 cache_hit 为 1,并进入 S_CACHE_RD 选择对应 word。

在未命中时,I-Cache 会先选择替换路。若当前 set 中存在无效 way,则优先选择无效 way,否则使用轮转指针。选择后将对应 valid 位置零,随后进入 S_MEM_RD 发起 32 Byte 对齐的内存读请求。在 S_RECV 状态中,Cache 每收到一个 beat,就根据 recv_cnt 写入 refill_line 的不同 32 bit 片段。等最后一个 beat 返回后进入 S_REFILL,写入 tag 和 data,并将 valid 位置为 1,最后从新写入的 refill line 中取出 CPU 所需的指令返回。

从仿真波形上看,命中情况下 CPU 请求只需要经历 tag 读取、cache 读取和响应几个状态;缺失情况下则会多出内存请求和多拍接收阶段。to_cpu_cache_rsp_valid 只在 S_RESP 状态拉高,保证 CPU 只在数据准备好后接收结果。

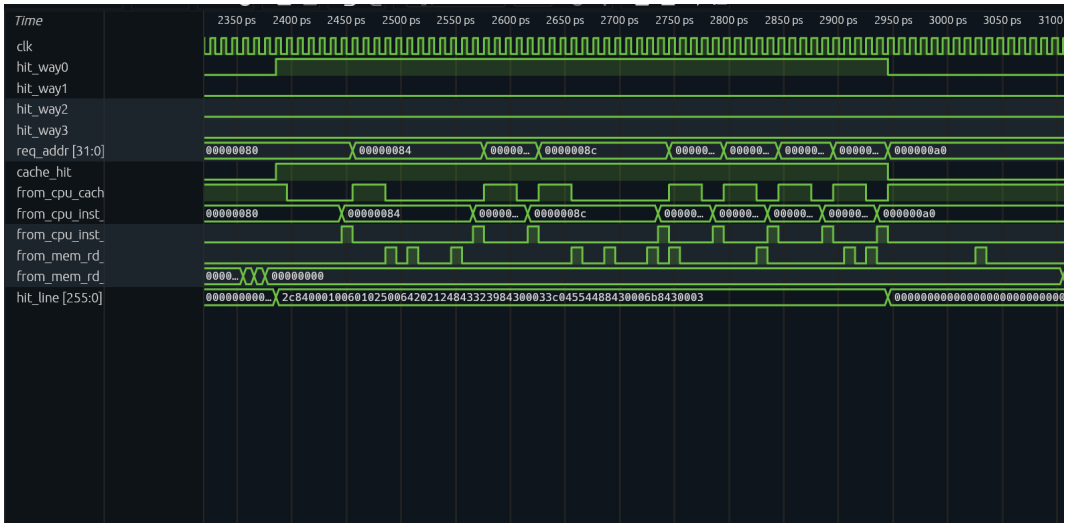


图 5: I-Cache 命中访问波形图

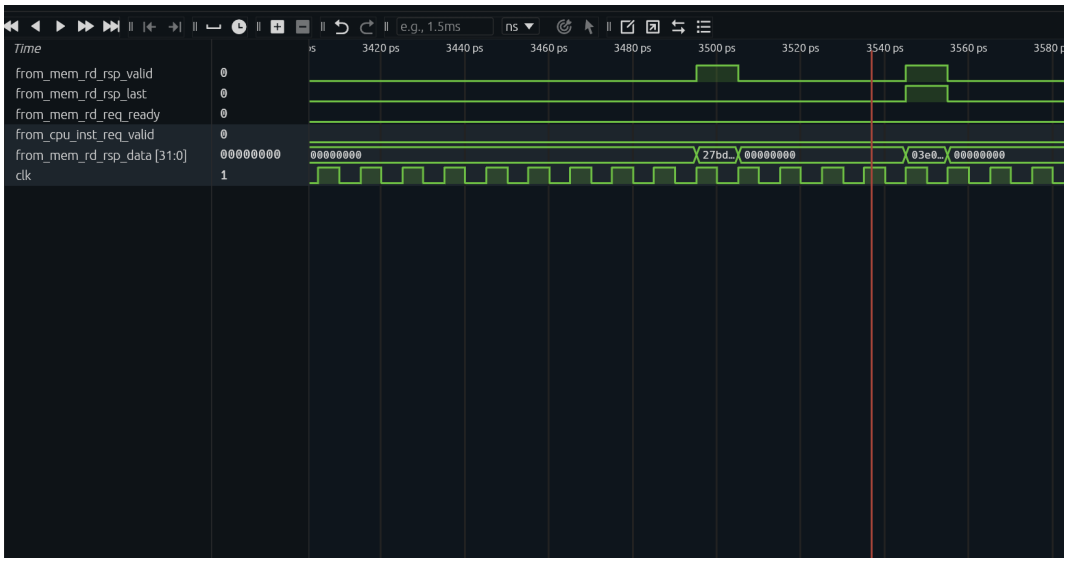


图 6: I-Cache 缺失后 burst refill 的波形图

3 D-Cache 开发

D-Cache 的开发重点在于写策略和状态机路径的完整性。相比 I-Cache，D-Cache 状态机额外加入了许多状态。这样可以分别处理脏行写回和不可缓存访问，避免把不同语义的访存请求混在同一条路径中。

对于写命中，我没有直接覆盖整个 32 bit word，而是先通过 merge_word 函数按字节选择旧数据或新数据，再通过 write_line 放回原 cache line。这样 CPU 传入的 wstrb 可以被完整保留下来。写命中完成后，D-Cache 不需要返回读数据，因此直接回到 S_WAIT，同时将对应 way 的 dirty 位置为 1。

对于读缺失和写缺失，D-Cache 都会先把整行数据从内存读入。不同之处在于，读缺失 refill 后需要选择目标 word 进入 S_RESP 返回 CPU；写缺失 refill 后需要先把写数据合并到新 cache line 中，然后直接回到 S_WAIT。这个设计让我对写分配策略有了更直观的理解：写缺失并不是只把一个 word 写到内存，而是把该地址所在的一整行纳入 Cache，后续相邻访问就可能直接命中。

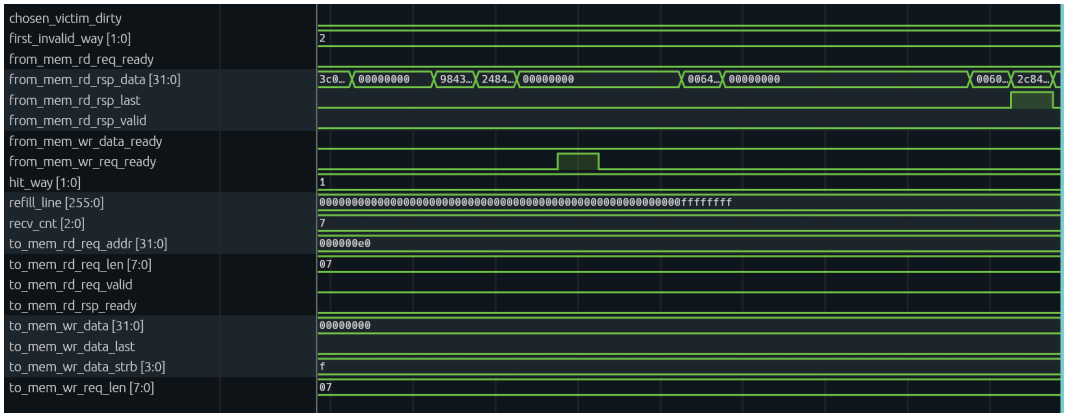


图 7: D-Cache 工作波形图

三、 学习总结(总结设计、开发与调试过程中遇到的问题,及实验中的收获与不足)

1 问题总结(实验过程中遇到的问题、对问题的思考过程及解决方法)

表 1: 实验问题总结

序号	问题类别	出错模块	问题现象	产生原因	解决办法
1	逻辑电路设计	I-Cache refill	缺失后返回的数据不是目标 word	接收 burst 数据时没有正确记录 word 在 cache line 中的位置	增加 recv_cnt ,按返回拍数写入 refill_line 的不同区间
2	逻辑电路设计	D-Cache 写命中	sb/sh 类写入会覆盖同一 word 中其它字节	起初只考虑了整字写入, 没有完整使用写掩码	增加 merge_word 和 write_line , 按 wstrb 逐字节合并
3	行为仿真	D-Cache bypass	外设地址被错误走入 Cache 流程	一开始没有把 I/O 地址和普通内存地址分开处理	对不可缓存地址增加 bypass 读写状态, 直接访问外部总线

- **Cache 控制逻辑问题补充**。

本次实验中最容易出错的地方是状态机和握手信号之间的关系。Cache 不是单纯的组合逻辑, 它既要面对 CPU 的请求/响应通道, 又要面对内存的读写请求和数据通道。如果在某个状态下提前拉高 ready 或 valid, 就可能导致请求被重复接收, 或者返回数据被提前消费。因此我在设计时尽量让每个状态只负责一类动作, 例如 S_MEM_RD 只发读请求,S_RECV 只接收读数据,S_RESP 只响应 CPU。这样虽然状态数量变多,但调试时更容易从波形中定位问题。

2 收获总结(在数字电路设计方法、RTL 编写方法、数字电路验证方法及其他方面的收获)

通过本次 Cache 实验,我对存储层次结构的理解更加具体。之前在理论课中学习 Cache 时,更多关注的是命中率、替换策略和平均访存时间等概念,而这次实验需要真正把 tag、valid、dirty、line、burst、bypass 和握手信号全部连接起来,才发现 Cache 实际上是一个相当复杂的控制器。尤其是 D-Cache 的写回和写分配策略,让我意识到一个简单的 store 指令背后可能包含读缺失、脏行写回、refill、字节合并等多个步骤。

本次实验的亮点在于 I-Cache 和 D-Cache 都保持了比较清晰的状态划分,四路组相联结构也通过数组模块进行了较好的复用;不足之处在于替换策略仍然比较简单,只使用了轮转指针,没有实现真正的 LRU。同时,当前验证主要依靠典型访存序列和波形观察,对极端情况下的连续缺失、bypass 与 cacheable 地址交错访问等覆盖还不够充分。

表 2: 收获总结

类别	名称	体会	后续计划
设计方法	四路组相联 Cache	理解了 tag/index/offset 的实际硬件划分，也掌握了 valid 和 dirty 位在不同 Cache 中的作用差异。	后续尝试实现更接近 LRU 的替换策略，并比较命中率差异。
RTL 编写	状态机和 burst 传输	能够用状态机拆分请求、等待、接收、回填和响应过程，但状态较多时信号命名仍需更规范。	将状态机输出信号进一步整理成独立组合逻辑，降低阅读和调试成本。
验证方法	波形调试	通过观察 valid/ready、state、recv_cnt 和 dirty 位变化，可以定位大部分 Cache 控制问题。	补充连续读写、脏行替换、I/O bypass 等专项测试用例。

四、 扩展思考(对讲义中思考题(如有)的理解和回答)

- **思考题 1:为什么 D-Cache 需要 dirty 位,而 I-Cache 不需要?**。

D-Cache 需要支持写请求。如果采用写回策略，CPU 写入的数据只会先保存在 Cache 中，并不会立即写到外部内存。因此 Cache 中的数据可能比内存中的数据更新，这时就需要 dirty 位记录该行是否被修改过。发生替换时，如果 dirty 位为 1，就必须先把旧行写回内存，否则修改会丢失。I-Cache 只负责取指，CPU 不会通过它修改指令内容，所以 Cache 中的数据始终来自内存，不存在比内存更新的问题，因此不需要 dirty 位。

- **思考题 2:为什么外设地址不应被普通 Cache 缓存?**。

外设寄存器和普通内存不同，它的值可能会被硬件随时改变，写入也可能代表一次实际动作。如果把外设地址缓存起来，CPU 后续读取到的可能只是 Cache 中的旧值，而不是硬件当前状态；连续写外设时也可能只修改 Cache 而没有真正发送到外设。因此这类地址需要通过 bypass 直接访问外部总线，保证每一次读写都真实发生。

五、 实验时长

在课后，你花费了大约 15 小时完成此次实验，其中各个阶段的用时如表 3所示。

步骤	时长(小时)	说明
设计	4	主要用于确定 Cache 参数、状态机和写回策略
开发	7	完成 I-Cache、D-Cache 和底层阵列模块
调试	4	通过波形检查命中、缺失、回填和 bypass 逻辑

表 3: 实验时长